

ROS_AIS_ws

ROS_AIS_ws

软件简介

- 软件设计目标
- 技术文件目标人群
- 技术文件涉及标准

软件使用说明

- 硬件要求
- 运行依赖
 - python库
 - ROS节点
- workspace部署
- 快速开始

软件整体设计

- 软件结构
- 常规运行过程
- 测试/开发运行过程
- 工作空间目录结构及文件功能
 - bag_files
 - ais_interfaces
 - ais_reports_interfaces
 - ais_nodes
 - ais_launch

ROS接口设计

- ais_interfaces
 - EPFSTimeStamp
 - ManeuverIndicator
 - NavigationStatus
 - TurnRate
 - VesselStaticInfoQuery
- ais_reports_interfaces
 - AISLocationReport
 - AISStaticAndVoyageReport

ROS节点设计

- ais_parse_node
 - 设计动机及实现
 - 接口
 - 回调函数设计
 - 节点参数
 - NMEA有限状态机实现
- ais_tf_node
 - 设计动机及实现
 - 接口
 - 回调函数设计
 - 节点参数
 - 投影算法实现
- ais_db_node
 - 设计动机及实现
 - 接口
 - 回调函数设计
 - 节点参数
- ais_map_pub_node
 - 设计动机及实现

接口
回调函数设计
节点参数
高效候选海图索引算法
矢量地图裁切，ENU转换优化算法
Launch文件设计

软件简介

软件设计目标

AIS信息是航行中重要的数据来源，它可以直接提供当前其它交通参与者的静态（包括目的地、ETA、几何大小等）及动态航行信息（包括操作状态、经纬度、速度等），对于实现辅助乃至自主的智能航行有较大价值。

ROS是一种开源的机器人系统框架，它包含了完善的通信机制、丰富的第三方节点以及众多传感器厂家的原生支持。无人艇作为一种水面机器人，亦可以采用ROS进行开发，并复用其中大量组件和算法，以达到事半功倍的效果，我司目前使用ROS2进行开发。

然而，AIS数据繁杂，其数据多为可视化后供有人航行使用，或科研使用，前者涵盖了大量的成熟航海电子仪器，比如船台上的集成AIS信息的电子海图单元，不适合于无人艇；后者则多为闭源代码，并不针对ROS架构，或者并不针对自主航行任务，或者兼而有之。

因此，本项目软件旨在基于ROS2，设计一套接口及节点，以接入AIS数据，服务本司无人艇产品的自主航行开发。

技术文件目标人群

ROS开发工程师，测试工程师，无人艇海试人员

技术文件涉及标准

IEC 61162-1 Maritime navigation and radiocommunication equipment and systems - Digital interfaces - Part 1: Single talker and multiple listeners

ITU-R M.1371-5 在VHF 水上移动频段内使用时分多址的自动识别系统的技术特性

软件使用说明

硬件要求

软件开发机器使用的硬件配置如下，该配置可视作稳定运行VMware Workstation Linux虚拟机+本软件的最低需求配置。

组件	最低配置	备注
处理器	i7-7920HQ	
内存	8 GB RAM	如果为实体机，此项要求可减少
存储	20 GB的额外可用空间	需包含虚拟机系统安装及ROS2软件包占用空间（建议使用SSD提升数据读写效率）
显卡	GTX 960M	如需运行机器视觉任务，建议配备高性能独立显卡（NVIDIA GTX 3060及以上）
操作系统	Ubuntu 22.04 LTS	ROS2 Humble Hawksbill官方支持版本，需64位系统
网络	以太网或Wi-Fi适配器	

运行依赖

python库

成功运行本软件需要包括但不限于以下列表的第三方库：

- transitions (<https://github.com/pytransitions/transitions>)
- pyais (<https://pyais.readthedocs.io/en/latest/>)
- numpy
- shapely (<https://shapely.readthedocs.io/en/stable/>)
- geopandas (<https://geopandas.org/en/stable/>)
- geographiclib (<https://geographiclib.sourceforge.io/>)

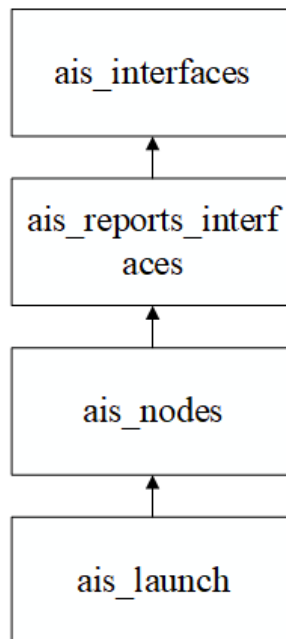
ROS节点

成功运行本软件需要包括但不限于以下列表的非官方ROS2包：

- nmea_navsat_driver (http://ros.org/wiki/nmea_navsat_driver)

workspace部署

与其它ROS2包相同，本项目需要 `source /opt/ros/[ros_version]/setup.zsh` 后，使用 `colcon build` 进行编译。为了避免编译问题，建议参考下图的依赖顺序使用 `--packages-select` 进行逐层编译：



参考编译指令如下：

```
# 在workspace目录下,假设zsh
source /opt/ros/[ros_version]/setup.zsh
colcon build --packages-select ais_interfaces
source ./install/setup.zsh
colcon build --packages-select ais_reports_interfaces
source ./install/setup.zsh
colcon build --packages-select ais_nodes ais_launch
```

快速开始

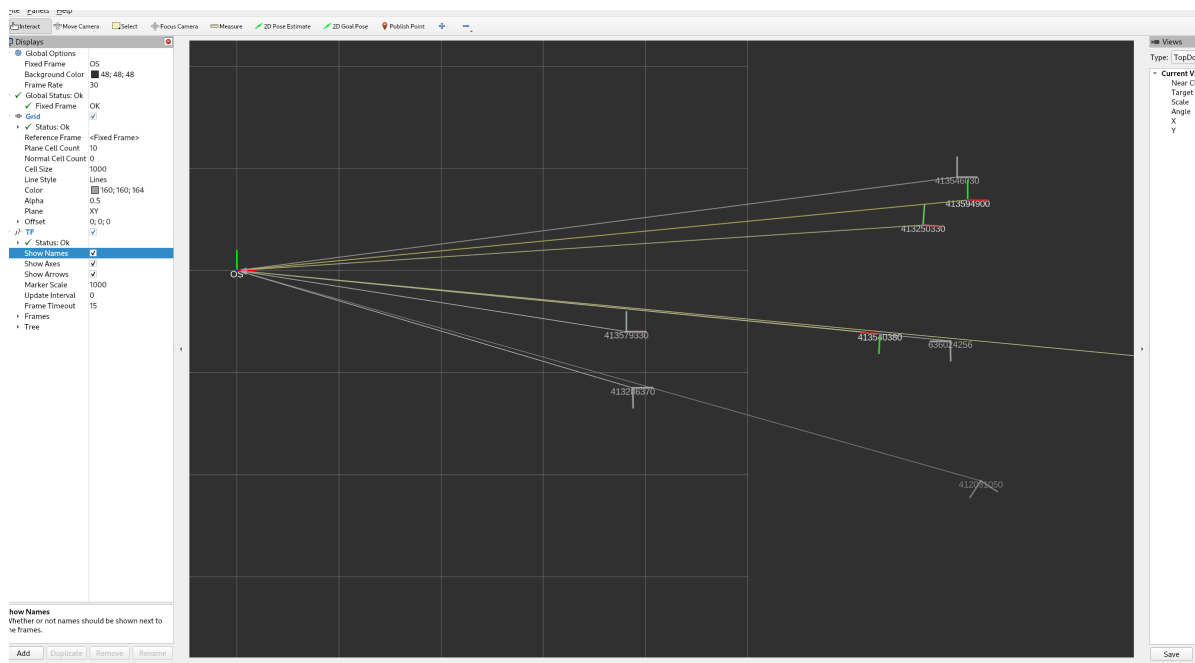
启用所有节点功能：

```
# 在workspace目录下,假设zsh,且workspace已完成build
source /opt/ros/[ros_version]/setup.zsh
source ./install/setup.zsh
ros2 launch ais_launch all_nodes.launch.py
```

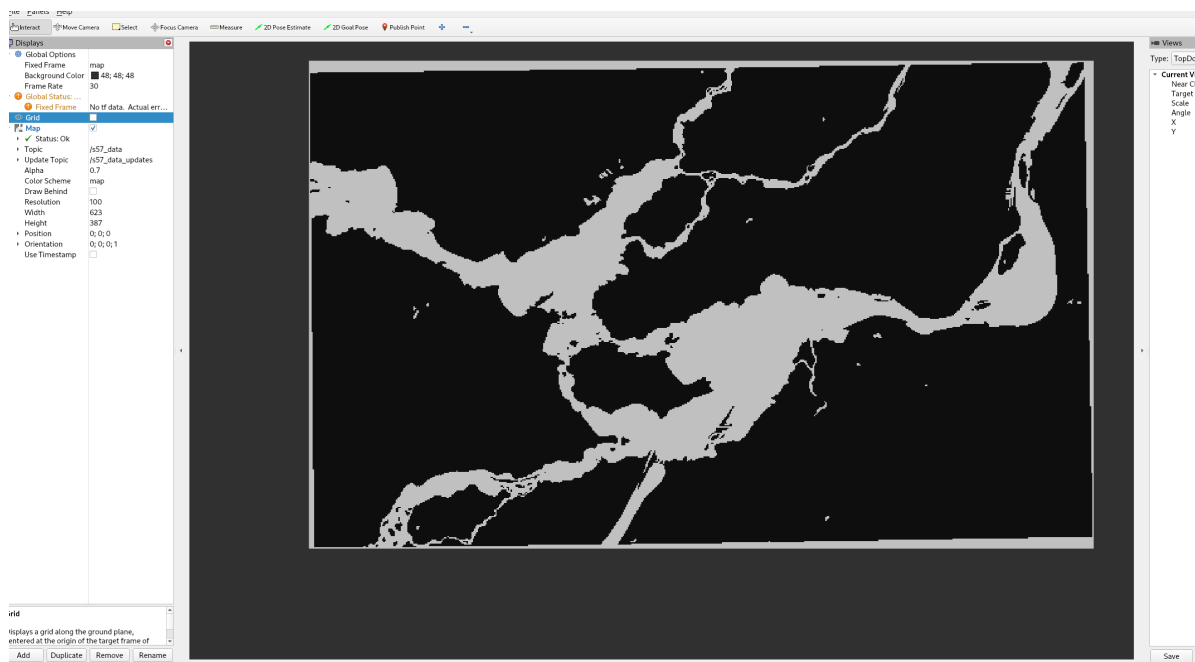
启用测试/开发配置：

```
# 在workspace目录下,假设zsh,且workspace已完成build
source /opt/ros/[ros_version]/setup.zsh
source ./install/setup.zsh
ros2 launch ais_launch setup_test.launch.py
bag_path:=/home/vectorwang/Workspace/ROS_AIS_ws/bag_files/type1235_and_nmea_record
```

成功启动后，开启本地 `rqt`，切换到TF视图可观察到以他船MMSI为名称的坐标信息，如图所示：

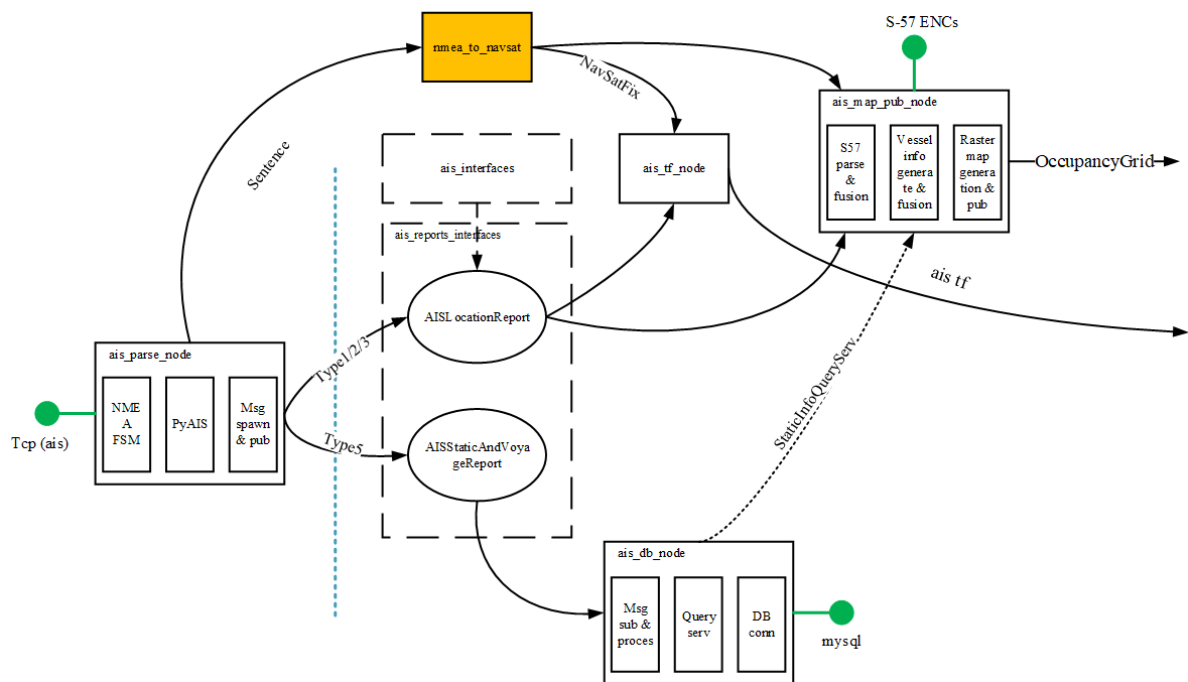


在Map视图可观察到以本船为中心，正方形视野范围内的栅格地图，如图所示：



软件整体设计

软件结构



目前的软件整体ROS2包和接口构成如图所示。其中，虚线框为工作空间内实现的ROS接口；实现框为ROS节点，其中橙色的为第三方节点；绿色圆形为工作空间运行依赖的外部接口。

ROS接口包包括：

- ais_interfaces
- ais_reports_interfaces

其中，ais_interfaces封装了枚举类型等，服务于ais_reports_interfaces的高级接口。

ais_reports_interfaces对本项目所用到的静态（5类AIS报文）及动态（1、2、3类AIS报文）进行了封装，服务于多种项目节点。

外部接口包括：

- AIS设备的TCP连接
- 本地存储的S-57海图及描述文件
- 基于Mariadb/Mysql的静态航行信息数据库

AIS设备的TCP连接为AIS设备提供，其功能、使用方法为，本地节点按照ip地址及端口连接AIS设备后，可以连续收到NMEA格式的AIS数据。包括明文数据（主要为本船信息），加密数据（主要为他船信息）。

本地存储的S-57海图及描述文件为本地文件，主要包括任务海域的S-57海图 .000 文件，以及预先计算的各海图覆盖文件 .json（设计详情请见[海图索引算法](#)相关章节）。

基于Mariadb/Mysql的静态航行信息数据库为预先在本地或云端部署的数据库，主要用于存储各船的静态航行信息，包括呼号、几何尺寸信息、吃水等。

ROS节点包括：

- ais_parse_node
- nmea_to_navsat（第三方）
- ais_tf_node
- ais_db_node
- ais_map_pub_node

其中，ais_parse_node主要作用为处理AIS原始信息，并将所需信息封装为ROS2消息进行发布。其中，明文信息主要涉及本船，封装为Sentence消息后交由nmea_to_navsat节点处理；密文信息主要涉及他船，封装为ais_reports_interfaces内定义的消息后发布，供本项目其它节点使用。

nmea_to_navsat节点为第三方节点，其作用为接收本船定位信息的NMEA明文消息，将其转换为ROS2标准的定位消息进行发布。

ais_tf_node节点作用为，根据本船定位数据及他船动态航行信息，发布他船至本船的相对坐标信息（TF）。

ais_db_node主要作用为管理静态航行信息数据库。其一是增改，即订阅来自ais_parse_node的静态消息，用来增加或修正数据库内存储的他船信息。其二是查，即提供StaticInfoQuery服务，其它节点可根据船舶MMSI，借由ais_db_node查询数据库内存储的他船静态信息。

ais_map_pub_node主要作用为提供本船周边区域的障碍物栅格地图。栅格地图标记的障碍物区域由海图记载的陆地块（包括岛屿等）、危险区域航标，及AIS通知的邻近他船构成。

常规运行过程

在具备AIS数据源时，项目软件发布坐标转换（TF）运行过程（数据流）大致如下：

1. ais_parse_node通过tcp连接获取AIS数据
2. 其中，明文和密文NMEA消息被分开处理，明文消息被封装为Sentence消息发布
3. 密文NMEA中，1~3类航行报文被封装为AISLocationReport消息发布，5类航行报文被封装为AISStaticAndVoyageReport发布
4. nmea_to_navsat订阅Sentence，发布标准的卫星定位报文NavSatFix等
5. ais_tf_node订阅NavSatFix及AISLocationReport，发布TF消息

项目软件发布邻域栅格地图（OccupancyGrid）运行过程（数据流）大致如下：

1. ais_parse_node通过tcp连接获取AIS数据
2. 其中，明文和密文NMEA消息被分开处理，明文消息被封装为Sentence消息发布
3. 密文NMEA中，1~3类航行报文被封装为AISLocationReport消息发布，5类航行报文被封装为AISStaticAndVoyageReport发布
4. nmea_to_navsat订阅Sentence，发布标准的卫星定位报文NavSatFix等
5. ais_db_node订阅AISStaticAndVoyageReport，更新数据库，并提供StaticInfoQuery服务
6. ais_map_pub_node订阅NavSatFix，使用本地S-57海图数据生成领域矢量地图
7. ais_map_pub_node订阅AISLocationReport，并调用StaticInfoQuery服务，动态维护领域他船的矢量障碍物图形
8. ais_map_pub_node融合上述信息，栅格化后，以OccupancyGrid消息形式发布栅格地图

测试/开发运行过程

测试/开发过程中，AIS数据源可能离线，因此需要调用历史数据进行回放。具体地，文中蓝色虚线以下的部分会被ROSBag数据回放所取代，而其它部分相同。

ROSBag回放的消息包括：

1. AISLocationReport
2. AISStaticAndVoyageReport
3. Sentence

其余过程与常规运行过程相同，此处不再赘述。

工作空间目录结构及文件功能

整个工作空间的文件、文件夹分布如下：

```
|
|---bag_files
|   |---type1235_and_nmea_record
|   |   |---metadata.yaml
```

```

| | type1235_and_nmea_record_0.db3
| |
| |
| | └type1235_record
| |     metadata.yaml
| |     type1235_record_0.db3
| |
| |
| └src
|     ├──ais_interfaces
|     |   ├──msg
|     |   |   EPFTimeStamp.msg
|     |   |   ManeuverIndicator.msg
|     |   |   NavigationStatus.msg
|     |   |   TurnRate.msg
|     |   |
|     |   └srv
|     |       VesselStaticInfoQuery.srv
|     |
|     ├──ais_launch
|     |   ├──ais_launch
|     |   |   __init__.py
|     |   |
|     |   ├──launch
|     |   |   all_nodes.launch.py
|     |   |   setup_test.launch.py
|     |   |
|     |   └resource
|     |
|     └test
|
|     ├──ais_nodes
|     |   ├──package.xml
|     |   ├──setup.cfg
|     |   ├──setup.py
|     |   |
|     |   ├──ais_nodes
|     |   |   ├──ais_db_node.py
|     |   |   ├──ais_map_pub_node.py
|     |   |   ├──ais_parse_node.py
|     |   |   ├──ais_tf_node.py
|     |   |   ├──__init__.py
|     |   |   |
|     |   |   ├──AIS_receiver
|     |   |   |   ├──ais_client.py
|     |   |   |   ├──fsm_nmea.py
|     |   |   |   ├──__init__.py
|     |   |   |
|     |   |   └geo_utils
|     |   |       ├──combine.py
|     |   |       ├──welzl.py
|     |   |       ├──__init__.py
|     |   |
|     |   └resource
|     |
|     └test
|
|     └ais_reports_interfaces
|         └msg
|             AISLocationReport.msg

```


bag_files

`bag_files` 为项目预先使用ROSBag录制的AIS数据，供离线开发及测试使用。Bag数据均在连云港海棠码头录制，录制时本船静止不动。

其中 `type1235_record` 文件夹包含了2024年12月5日录制的15分钟历史数据，数据内容包括 `ais_parse_node` 发布的 `AISLocationReport` 及 `AISStaticAndVoyageReport` 消息。

`type1235_and_nmea_record` 文件夹包含了2024年12月10日录制的15分钟历史数据，数据内容包括 `ais_parse_node` 发布的 `AISLocationReport`、`AISStaticAndVoyageReport` 以及 `Sentence` 消息。其中 `Sentence` 消息为本船AIS设备发出明文NMEA消息，主要内容为本船定位信息等。

ais_interfaces

`ais_interfaces` 为接口定义包，不包含节点。

`ais_interfaces` 共定义四种消息，一种服务接口。消息接口文件保存于 `msg` 文件夹中，服务接口文件保存于 `srv` 文件夹中。

其中4种消息均为特定AIS段信息的封装，服务于 `ais_reports_interfaces` 定义的相关接口。服务接口定义了查询数据库中船舶静态信息的接口格式，该服务由 `ais_db_node` 提供，在本仓库提供的节点中，由 `ais_map_pub_node` 调用。

接口内容请参阅接口设计章节。

ais_reports_interfaces

`ais_reports_interfaces` 为接口定义包，不包含节点。

`ais_reports_interfaces` 定义了两类航行相关AIS消息的封装（消息接口），均保存于 `msg` 文件夹中。其中 `AISLocationReport.msg` 为第1、2、3类AIS航行报文的ROS封装，`AISStaticAndVoyageReport.msg` 为第5类静态与航行信息报文的ROS封装。

相关接口定义见接口设计章节，AIS报文内容和规范见本文开头给出的相关标准。

ais_nodes

`ais_nodes` 为节点定义包，所有节点源代码均为python格式，储存于 `ais_nodes` 文件夹中。

其中，`ais_db_node.py`、`ais_map_pub_node.py`、`ais_parse_node.py`、`ais_tf_node.py` 四个py文件为节点的源码，分别用于静态航行信息数据库管理和服务、栅格地图发布、AIS数据解析和发布、他船坐标信息发布功能，详情见节点设计章节。

`AIS_receiver` 文件夹为节点使用的，AIS解析相关库，包括NMEA有限状态机、AIS客户端、解析代码等相关功能，详情见节点设计章节。

`geo_utils` 文件夹为节点使用的，地理信息操作相关库，包括相关海图索引、海图拼接等相关功能，详情见节点设计章节。

ais_launch

`ais_launch` 为launch文件定义包，不包含节点。

项目workspace默认提供两种launch文件，均包含在 `launch` 文件夹内。其中，`all_nodes.launch.py` 针对海试等AIS实际运行的工况设计，将启用所有可用功能。`setup_test.launch.py` 针对离线开发、测试等AIS设备不在线的工况设计，此时将使用ROS Bag调用历史数据进行回放，发布AIS消息及NMEA明文报文，以模拟真实AIS的测试环境。

相关launch文件详情见launch文件设计章节。

ROS接口设计

ais_interfaces

EPFSTimeStamp

EPFSTimeStamp为电子定位系统（EPFS）生成报告的时间（0-59，单位秒），及时戳状态。消息包含以下数据：

参数	类型	备注
timestamp_status	uint8	时戳状态
seconds	uint8	报告生成时间

时戳状态为以下取值之一：

状态	枚举值	备注
STATUS_NORMAL	0	状态正常
TIME_STAMP_NOT_AVAILABLE	1	时戳不可用
SYSTEM_MANUAL_INPUT_MODE	2	定位系统人工输入模式
EPFS_DEAD_RECKONING_MODE	3	定位系统航迹推算模式
SYSTEM_INOPERATIVE	4	定位系统不起作用

ManeuverIndicator

ManeuverIndicator为特定操纵指示符。消息包含以下数据：

参数	类型	备注
maneuver_indicator	uint8	操纵状态

操纵状态为以下取值之一：

状态	枚举值	备注
MANEUVER_NOT_AVAILABLE	0	不可用（默认值）
NO_SPECIAL_MANEUVER	1	未进行特定操纵
SPECIAL_MANEUVER	2	进行特定操纵

NavigationStatus

NavigationStatus为导航状态。消息包含以下数据：

参数	类型	备注
navigation_status	uint8	导航状态

导航状态为以下取值之一：

状态	枚举值	备注
UNDER_WAY_USING_ENGINE	0	发动机使用中
AT_ANCHOR	1	锚泊
NOT_UNDER_COMMAND	2	未操纵
RESTRICTED_MANOEUVRABILITY	3	有限适航性
CONSTRAINED_BY_HER_DRAUGHT	4	受船舶吃水限制
MOORED	5	系泊
AGROUND	6	搁浅
ENGAGED_IN_FISHING	7	从事捕捞
UNDER_WAY_SAILING	8	航行中
RESERVED_FOR_FUTURE_AMENDMENT_OF_NAVIGATIONAL_STATUS_FOR_HSC	9	留做将来修正导航状态
RESERVED_FOR_FUTURE_AMENDMENT_OF_NAVIGATIONAL_STATUS_FOR_WIG	10	留做将来修正导航状态
POWER_DRIVEN_VESSEL_TOWING_ASTERN	11	机动船尾推作业（区域使用）
POWER_DRIVEN_VESSEL_PUSHING_AHEAD_OR_TOWING_ALONGSIDE	12	机动船顶推或侧推作业（区域使用）
RESERVED_FOR_FUTURE_USE	13	留用
AIS_SART_IS_ACTIVE	14	AIS-SART（现行）、MOB-AIS、EPIRB-AIS
UNDEFINED	15	未规定（默认值）

TurnRate

TurnRate为船舶转向率及转向状态状态。消息包含以下数据：

参数	类型	备注
turn_status	uint8	转向状态
turn_rate	float32	转向率，单位：度/分钟，正值为右转，负值为左转

转向状态为以下取值之一：

状态	枚举值	备注
TURN_STATUS_OK	0	外部旋转率指示器 (TI) 状态正常
TURNING_RIGHT_RATE_EXCEEDING_RANGE_NO_TI_AVAILABLE	1	以每30秒右旋超过5度的速率旋转 (TI不可用)
TURNING_LEFT_RATE_EXCEEDING_RANGE_NO_TI_AVAILABLE	2	以每30秒左旋超过5度的速率旋转 (TI不可用)
NO_TURN_INFORMATION_AVAILABLE	3	没有可用的旋转信息 (默认值)

VesselStaticInfoQuery

VesselStaticInfoQuery用于请求/提供船舶的静态信息。请求包含以下数据：

参数	类型	备注
mmsi	uint32	船舶识别号

返回数据包含以下数据：

参数	类型	备注
mmsi	uint32	船舶识别号，返回数据的mmsi与请求相同
available	bool	是否查询到该船信息
vessel_name	string<=20	船舶名称
to_bow	uint16	定位基准点到船艏距离
to_stern	uint16	定位基准点到船艉距离
to_port	uint16	定位基准点到左舷距离
to_starboard	uint16	定位基准点到右舷距离
draught	float32	目前最大静态吃水，单位米，0=不可用=默认值

ais_reports_interfaces

AISLocationReport

AISLocationReport用于封装船舶的动态航行信息，包括1、2、3类AIS航行报文。消息包含以下数据：

参数	类型	备注
timestamp	builtin_interfaces/Time	ROS时间戳，时间为该消息在节点内的生成时间
msg_type	uint8	AIS航行报文类型，取值为1、2、3
repeat	uint8	消息被转发次数，供AIS转发器使用
mmsi	uint32	船舶唯一标识符（但也可能为其它用户ID）
navigation_status	ais_interfaces/NavigationStatus	导航状态
turn_rate	ais_interfaces/TurnRate	旋转速率
is_valid_sog	bool	地面航速是否可用
sog	float32	地面航速
location_accuracy	bool	位置准确度，该项定义详情见ITU-R M.1371-5的表50
is_valid_longitude	bool	经度是否可用
longitude	float32	经度值，其中东为正值，西为负值
is_valid_latitude	bool	纬度是否可用
latitude	float32	纬度值，其中北为正值，南为负值
is_valid_cog	bool	对地航线方向是否可用
cog	float32	对地航线方向，单位度
is_valid_hdg	bool	艏向是否可用
hdg	int16	艏向，单位度
epfs_time_stamp	ais_interfaces/EPFSTimeStamp	EPFS时间戳
maneuver_indicator	ais_interfaces/ManeuverIndicator	特定操纵指示符
raim	bool	电子定位装置自主整体检测（RAIM）标志，该项定义详情见ITU-R M.1371-5的表50
radio	uint32	通信状态，该项定义详情见ITU-R M.1371-5的表49

AISStaticAndVoyageReport

AISStaticAndVoyageReport用于封装船舶的静态航行信息，包括5类AIS航行报文。消息包含以下数据：

参数	类型	备注
timestamp	builtin_interfaces/Time	ROS时间戳，时间为该消息在节点内的生成时间
msg_type	uint8	AIS航行报文类型，取值为1、2、3
repeat	uint8	消息被转发次数，供AIS转发器使用
mmsi	uint32	船舶唯一标识符（但也可能为其它用户ID）
ais_version	uint8	AIS版本指示符，该项定义详情见ITU-R M.1371-5的表52
imo_number	uint32	IMO编号
call_sign	string<=7	呼号，7位ASCII字符，@@@@@@=不可用=默认值
vessel_name	string<=20	船舶名称
ship_type	uint8	船舶和货物类型，0 = 不可用或没有船舶 = 默认值，该项定义详情见ITU-R M.1371-5的3.3.2节
to_bow	uint16	定位基准点到船艏距离
to_stern	uint16	定位基准点到船艉距离
to_port	uint16	定位基准点到左舷距离
to_starboard	uint16	定位基准点到右舷距离
epfd_fix_type	uint32	电子定位装置的类型，该项定义详情见ITU-R M.1371-5的表52
eta_month	uint8	预计到到时间，月
eta_day	uint8	预计到到时间，日
eta_hour	uint8	预计到到时间，时
eta_minute	uint8	预计到到时间，分
draught	float32	目前最大静态吃水，单位米，0=不可用=默认值
destination	string<=20	目的地
dte	bool	数据终端是否就绪，该项定义详情见ITU-R M.1371-5的3.3.1节

ROS节点设计

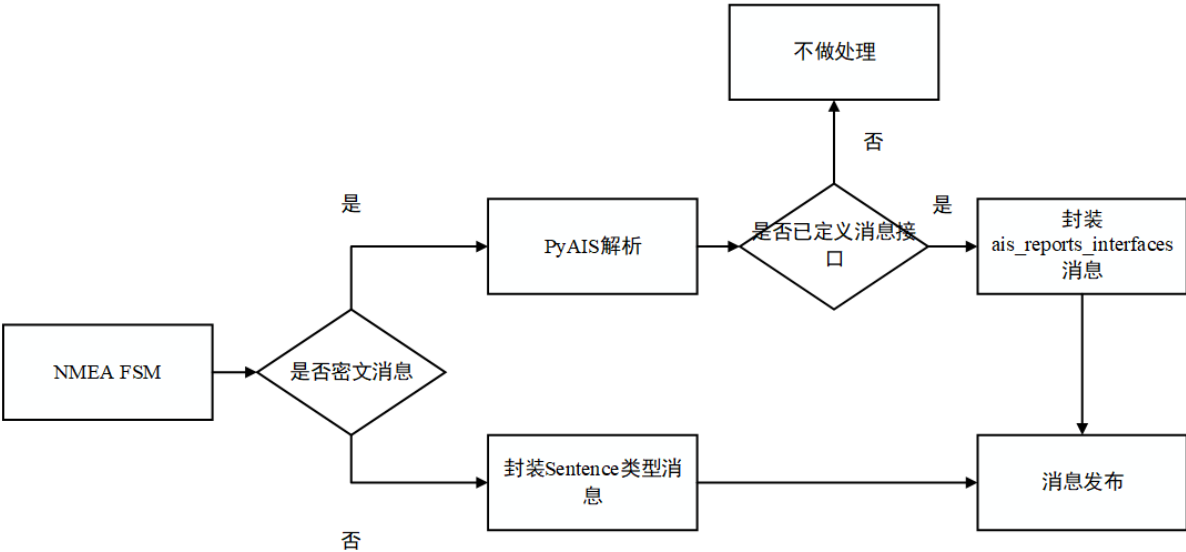
ais_parse_node

设计动机及实现

在实现坐标信息及栅格地图输出之前，需要首先对AIS接口传入的原始数据进行处理，以将其转变为ROS2内的消息形式。考虑到以下设计需求：

- 1. AIS传出的NMEA消息包含两类，明文消息主要为本船信息，密文消息主要为他船信息，其处理流程各不相同
- 2. NMEA消息存在沾包及误码的可能性，需要机制保障信息处理的可靠性
- 3. 目前只需要特定几种类型的AIS报文，但对AIS信息的处理需要考虑未来的拓展性，扩展功能最好只涉及少数节点
- 4. AIS设备并非时常能够远程调试，需要考虑搜集历史数据进行离线调试的可能性

因此，本项目中采用单个节点ais_parse_node对AIS接口数据进行分类、简单处理、过滤和发布，其数据处理流程如下图所示：



采用这种设计有以下好处：

- 1. 使用有限状态机，可以良好应对沾包等传输异常
- 2. 拓展性好，如果需要处理新种类的密文消息仅需添加对应的msg类型，并更改本节点即可实现发布
- 3. 仅本节点负责与AIS接口交互，同时借助ROS2消息发布机制，实现了对AIS接口的隔离，完成松耦合紧内聚的设计
- 4. 仅本节点负责与AIS接口交互，如果需要离线调试或开发，可以使用ROSBag仅对本节点发布的数据进行录制

接口

ais_parse_node节点没有ROS2输入接口，但有以下输出接口：

接口名称（默认值）	接口类型	备注
ais_location_report	ais_reports_interfaces/msg/AISLocationReport	AIS 1、2、3类报文
ais_static_and_voyage_report	ais_reports_interfaces/msg/AISStaticAndVoyageReport	AIS 5类报文
nmea_sentence	nmea_msgs/msg/Sentence	明文NMEA

回调函数设计

ais_parse_node节点没有定时器、服务调用或消息订阅，因此没有ROS2意义的回调函数。

但是节点的密文NMEA消息处理使用AIS_receiver.ais_client.AIS_Client对象，该对象的设计有类似回调的机制，开发者可以使用

```
AIS_receiver.ais_client.AIS_Clientupdate_encrypted_handlers([msg_type],
[handler_func])
```

将AIS报文类型与处理函数进行绑定，详情可见该方法的源代码。

节点参数

ais_parse_node有以下节点参数：

参数名称	参数类型	参数默认值	备注
ais_ip	String	192.168.254.40	AIS设备的ip地址
ais_port	Integer	8004	AIS设备的端口
gps_nmea_topic	String	nmea_sentence	明文NMEA的消息名称

NMEA有限状态机实现

动机：

在基于NMEA协议的串口通信中，沾包（Packet Sticking）是常见的数据流异常现象，表现为接收端因数据帧边界缺失导致多个报文粘连。例如，GPS模块连续发送的 \$GPRMC 和 \$GPGGA 语句若未通过分隔符（如换行符 \r\n）明确分割，可能被合并为单一数据块，引发解析错位或校验失败。其成因通常源于底层传输层（如UART或TCP）的缓冲区机制、高频率数据发送或硬件中断延迟。

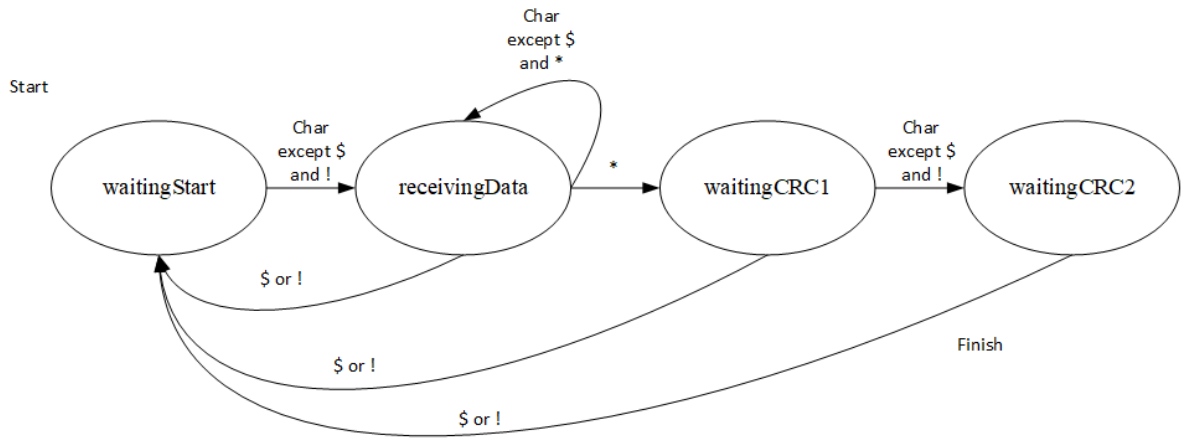
有限状态机（Finite State Machine, FSM）通过定义离散状态与状态转移规则，可有效解决沾包问题。其核心原理为：

- 1. 状态划分：根据NMEA协议规范（如起始符\$、字段分隔符,、校验和*及终止符\r\n），将解析过程分解为 等待起始符、读取数据体、校验验证 等状态；
- 2. 逐字符处理：逐个字节驱动状态转移，动态识别报文边界，避免依赖固定长度或分隔符预判；
- 3. 容错恢复：通过超时机制或错误状态回退，丢弃无效数据并重置至初始状态，保障后续报文解析可靠性。

相较于传统缓冲区分割算法，FSM通过逻辑状态显式管理解析进度，即使数据流存在沾包或碎片化，仍能精准提取完整报文，显著提升通信鲁棒性。

FSM实现：

本文的有限状态机基于python transitions 库，采用如下的结构设计：



算法性能:

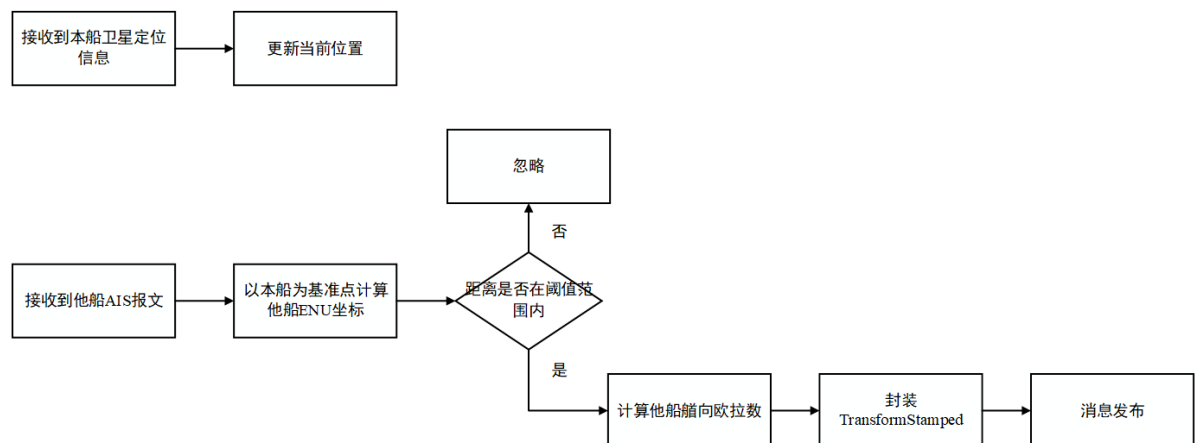
时间复杂度为 $O(n)$ ，其中n为缓冲区字符数量。

ais_tf_node

设计动机及实现

在ROS2中，TF (Transforms) 库是机器人系统中实现多坐标系动态关系管理的核心工具。其通过维护基于时间戳的坐标变换树 (TF Tree)，实时计算同系统不同机器人间，传感器、执行机构与机器人本体之间的位姿关系（如平移与旋转），并支持跨节点、跨时间戳的坐标查询（即“时间旅行”查询）。TF2作为ROS2的底层库，优化了数据结构和线程安全机制，支持四元数、欧拉角等多种姿态表示方式，并可通过 `tf2_ros` 工具包与ROS2消息系统深度集成。在导航、机械臂运动规划等场景中，TF通过消除多坐标系数据融合时的坐标歧义，确保空间数据一致性，同时为模块化系统设计提供统一的坐标参照基准，是构建复杂机器人系统的重要基础设施。

在本软件中，通过TF形式表征本船与他船的态势信息。通过将GPS信息统一到本船坐标系下的相对位置，为自主航行提供数据支撑。ais_tf_node接收本船和他船航行信息作为输入，以标准的TF消息作为输出，其基本运行过程如下：



接口

ais_tf_node节点订阅以下消息作为输入：

接口名称（默认值）	接口类型	备注
ais_location_report	ais_reports_interfaces/msg/AISLocationReport	AIS航行信息报文
fix	sensor_msgs/msg/NavSatFix	本船卫星定位信息

ais_tf_node输出以下信息：

[他船MMSI]至[本船名称]的TransformStamped。

回调函数设计

ais_tf_node包含两个回调函数：

- ais_location_cb, ais_location_report消息触发的他船航行数据处理函数
- self_gps_cb, fix消息触发的本船卫星定位数据处理函数

节点参数

ais_tf_node有以下节点参数：

参数名称	参数类型	参数默认值	备注
os_name	String	OS	定义了本船在ROS2的tf坐标中显示为什么名称
filter_threshold_km	Double	10.0	仅考虑该半径范围内的他船数据

投影算法实现

动机：

AIS传输的坐标数据为来自GPS、北斗等全球定位系统的经纬度坐标，而ROS2内，传输tf数据、栅格地图数据，需要XYZ形式的直角坐标。因此，需要使用投影算法，将经纬度数据转为直角坐标形式。

主流的投影方法包括UTM、墨卡托、直接线性变换等。考虑到本项目中对相对方位和距离敏感，且任务区域相对较小（仅针对本艇周围数公里至十数公里海域），上述方法均不适用：UTM转换复杂，且要考虑跨区问题，墨卡托和直接经纬度转换带来方位和距离偏差大。

算法实现：

基于球体几何，以本船坐标为基准点，计算各目标经纬度相对本船的角度及球面距离，然后由极坐标转换为ENU（East-North-Up）直角坐标。

早期版本中，投影算法为自行实现（基于Vincenty公式等），目前版本为基于第三方库geographiclib实现，来保证转换精度和可靠性。

算法性能：

时间复杂度为 $O(n)$ ，其中n为待转换特征数量。

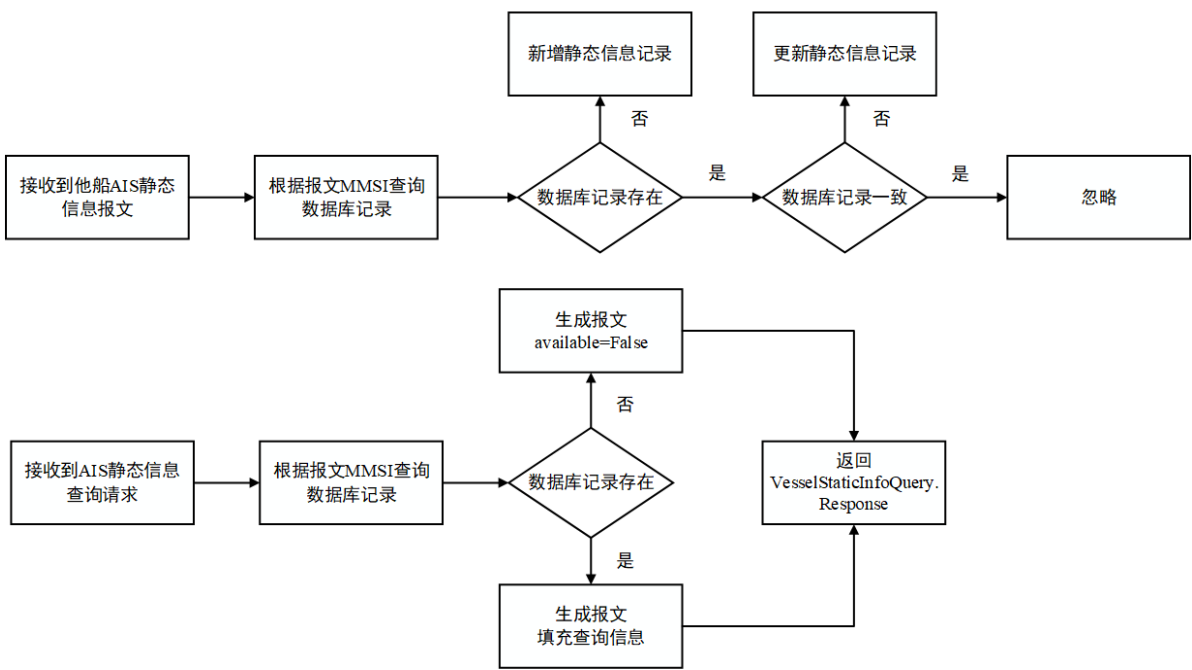
ais_db_node

设计动机及实现

他船的静态航行信息，如AIS数据给出的卫星定位的基准点、船舶的吃水深度、船舶的长宽等静态航行信息，往往是保持不变的。且AIS 5类报文往往发送频率较低，如果每次软件开启后都需要重新获取静态信息，往往态势构建的实时性会很差，所以本项目中设计了静态航行信息数据库对这些信息进行长效存储，供有关节点使用。考虑到以下设计需求：

1. 数据库结构可能改变，比如未来可能增加ETA、目的地等相对动态的信息
2. 有多个节点均需要从数据库获取静态航行信息
3. 新获取的静态航行信息报文需要解读，并根据需要更新到数据库中

如果有关节点均存储数据库访问密钥，与数据库直接交互，无疑是效率低、安全性差的。此外，数据库的结构、访问方式改变，会导致所有有关节点均需要变动代码。因此，本项目以“封装变化”为目的，设计了ais_db_node单独负责与数据库进行交互，并对外提供统一的ROS2查询服务。ais_db_node的数据处理流程如下图所示：



接口

ais_db_node节点订阅以下消息作为输入：

接口名称	接口类型	备注
ais_static_and_voyage_report	ais_reports_interfaces/msg/AISStaticAndVoyageReport	AIS静态航行信息报文

ais_db_node提供以下查询服务：

接口名称	接口类型	备注
vessel_static_info_query	ais_interfaces/srv/VesselStaticInfoQuery	AIS船舶静态信息查询服务

回调函数设计

ais_db_node包含两个回调函数：

- static_and_voyage_subscriber_cb, ais_static_and_voyage_report消息触发的他船静态航行数据处理函数
- static_info_service_cb, vessel_static_info_query服务的响应函数

节点参数

无，因为默认数据库部署后设置固定，所以相关参数在源代码中进行定义，没有设计为节点参数。

ais_map_pub_node

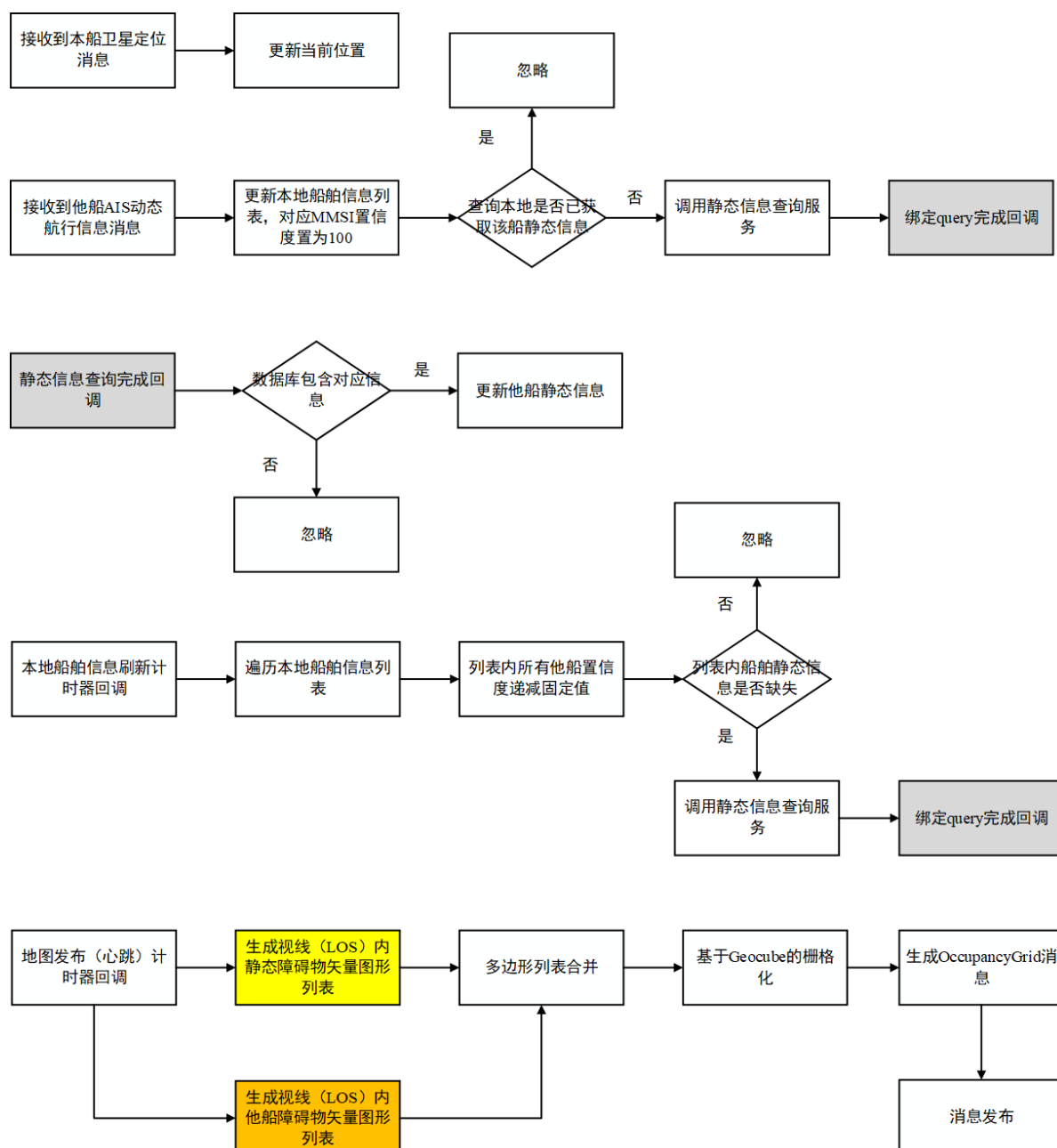
设计动机及实现

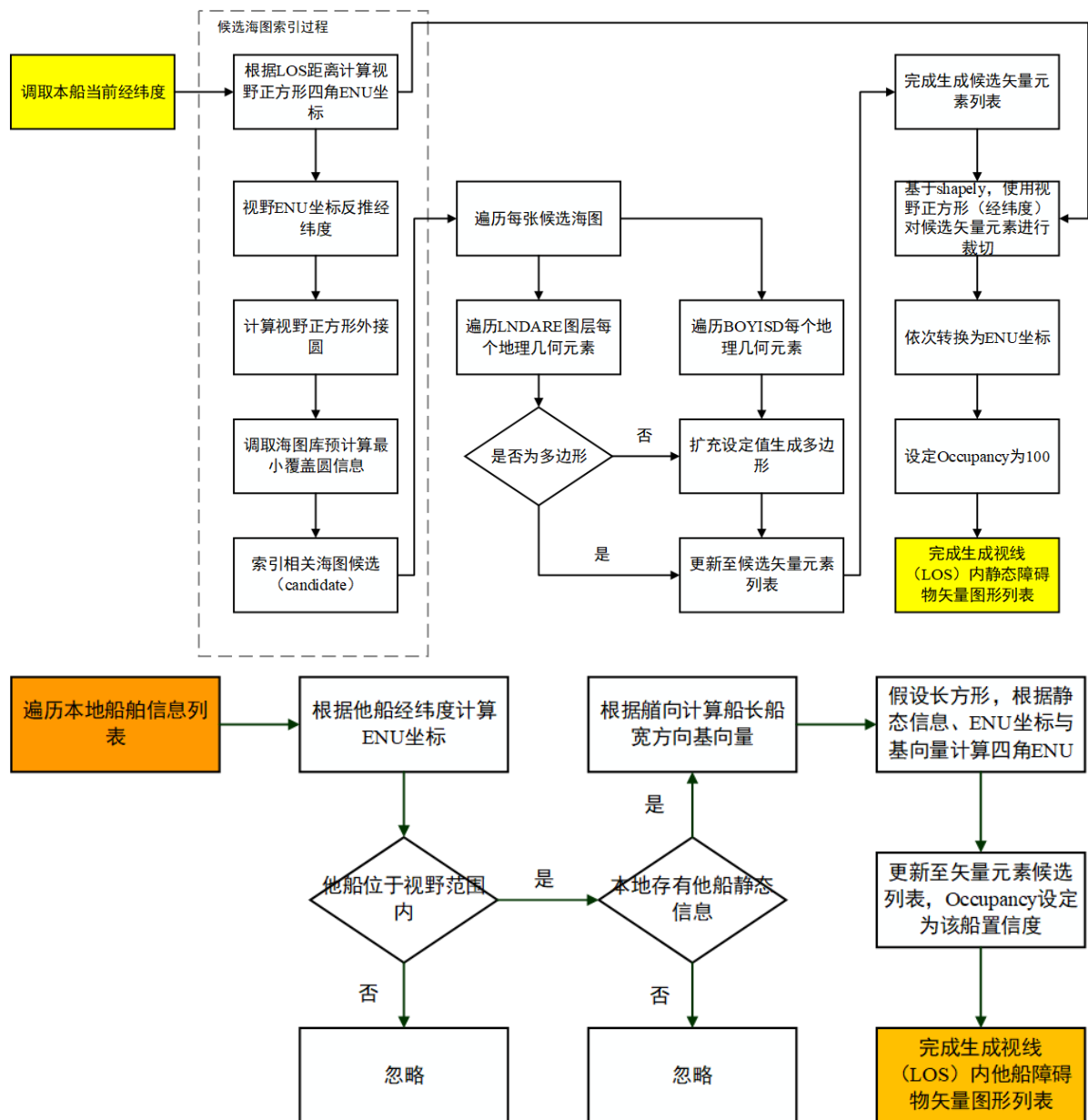
在无人艇的自主导航系统中，S57海图提供了丰富的先验静态信息，而AIS提供了关于其它交通参与者的，较为实时的复杂动态信息。然而，上述提到的信息，往往不能为路径规划算法直接使用。路径规划往往依赖于局部地图，而一张典型的S57海图包含十数乃至数十个不同图层，其几何元素信息从三维点、样条线到多边形不等，需要提取所需信息加以利用；AIS报文则为有关信息的列表，不直接包含几何元素。因此，如何利用S57和AIS信息，融合生成本船领域的栅格地图以供自主导航参考，成为开发难点。

进一步地，栅格地图的生成还需要考虑以下技术问题：

1. 经纬度到本地直角坐标系的批量转换问题（ais_tf_node部分解决，本节后续介绍）
2. 本船邻近区域的相关海图索引问题（本节后续介绍）
3. 他船态势的构建和刷新机制问题（目前采用定时衰减无新报文的他船的可信度的做法）

应对上述开发需求与问题，设计开发了ais_map_pub_node节点，其工作流程如下图所示：





接口

ais_map_pub_node节点使用以下信息作为输入:

接口名称 (默认值)	接口类型	备注
ais_location_report	ais_reports_interfaces/msg/AISLocationReport	AIS航行信息报文
fix	sensor_msgs/msg/NavSatFix	本船卫星定位信息
vessel_static_info_query	ais_interfaces/srv/VesselStaticInfoQuery	AIS船舶静态信息查询服务

ais_map_pub_node发布以下消息:

接口名称 (默认值)	接口类型	备注
s57_data	nav_msgs/msg/OccupancyGrid	本船附近的栅格地图

回调函数设计

ais_map_pub_node包含三个回调组，五个回调函数：

- 1. 默认回调组（互斥），包含消息订阅回调函数
 - gps_cb, fix消息触发的本船卫星定位数据处理函数
 - ais_location_cb, ais_location_report消息触发的他船航行数据处理函数
- 2. 互斥组1，包含定时器的回调函数，采用互斥组的原因，两个回调函数均会更新本地栅格地图，导致资源竞争
 - timer_cb, 定时栅格地图发布
 - refresh_vessel_static_cb, 定时衰减他船在栅格地图上的可信度
- 3. 并行组1，包含为请求服务获得响应的回调函数
 - query_done_cb, 在获得VesselStaticInfoQuery.Response后，更新本地的他船静态航行信息

节点参数

ais_map_pub_node有以下节点参数：

参数名称	参数类型	参数默认值	备注
s57_dir	String	/	S57海图文件所在目录
s57_coverage	String	/	S57海图覆盖区域JSON位置
resolution	Double	10.0	栅格化后的分辨率，单位米
los_distance	Integer	5000	视野范围，以本船为中心，该长度*2为边长的正方形区域为最后输出的邻居区域，单位米
buffer_size	Double	5.0	对于点状障碍物扩充该半径长度作为危险区域，单位米
decay_per_period	Integer	20	他船长时间无报文情况下，其栅格可信度的衰减速度
pub_period	Integer	10	栅格地图发布周期，单位秒
map_topic	String	s57_data	栅格地图使用的话题名称
map_name	String	map	栅格地图在TF中的名称
os_name	String	OS	本船在TF中的名称

高效候选海图索引算法

动机：

海图库中，中国海域的海图有数十至上百张。每张海图精度、覆盖范围各不相同。而本船附近区域（任务海域）范围有限，往往只涉及单张或数张相关海图。如何快速索引出相关海图，需要开发算法实现。

直接实现与局限：

海图的覆盖范围为多边形，一种直接的思路是读取海图的M_COVR图层，然后每张海图依次与任务区域求交集，若交集不为空则对应海图可能包含任务海域所需信息。

经过实测发现，该方法性能不能满足要求。viztracer抓取运行数据发现，主要性能瓶颈为海图M_COVR读取（测试机约花费30秒），所有海图的交集求取（花费数百毫秒）。即使将M_COVR长期载入内存节省每次读取时间，时间花费仍差强人意。

算法设计思路：

以存储换运算效率，以精度换算法速度。

预先求取每个海图覆盖范围的最小外接圆（使用Welzl算法），存储本地数据库中。索引相关海图时，先求任务海域的最小外接圆，然后用该圆与数据库中预置圆进行比较，此时求交集转为求圆心距离，再比较与半径和大小，计算效率极大提高。

索引有关海图的伪代码如下：

```
function find_s57_within_range(os_longi, os_lati, los):
    s57_candidates = []

    # 目标区域为正方形，求对角线长度的一半作为实际半径
    os_r = los * np.sqrt(2)

    # 对每张s57海图，求是否可能与本船领域相交
    for each s57_map:
        area_longi, area_lati, area_r = get_coverage(s57_map)
        dist = distance(os_longi-area_longi, os_lati-area_lati)
        if dist <= (os_r + area_r):
            s57_candidates.append(s57_map)

    return s57_candidates
```

其中，Welzl算法为经典的递归求外接圆算法，其伪代码如下：

```
# 初始输入points为全部点集，对多边形而言为所有外轮廓点
function welzl(points, support):
    if len(points) == 0 or len(support) == 3:
        return min_circle_from_points(support)

    # 随机选择一个点
    p = random.choice(points)

    # 递归处理剩余点
    D = welzl(points - {p}, support)

    if p is inside(D):
        return D
    else:
        return welzl(points - {p}, support ∪ {p})

function min_circle_from_points(points):
    if len(points) == 0:
        return None # 空集没有包围圆
    elif len(points) == 1:
        return Circle(center=points[0], radius=0)
    elif len(points) == 2:
        return circle_from_two_points(points[0], points[1])
    elif len(points) == 3:
        # 检查是否存在三个点中的两点组成的圆包含第三个点
        for i in 0 to 2:
```



```

        c = circle_from_two_points(points[i], points[(i+1)%3])
        if c.contains(points[(i+2)%3]):
            return c
    # 否则返回三个点的外接圆
    return circumcircle(points[0], points[1], points[2])

# 辅助函数
function circle_from_two_points(a, b):
    center = midpoint(a, b)
    radius = distance(a, b) / 2
    return Circle(center, radius)

function circumcircle(a, b, c):
    # 计算三个点的外接圆（实现细节略）
    # 可通过垂直平分线交点求解
    return Circle(...)

```

算法性能：

测试机上，算法实际运行花费数毫秒至数十毫秒，性能提升较大，满足工程需求。缺陷为，可能筛选出多余的候选海图。

未来改进：

考虑到大部分海图仍为长方形或近似长方形，可以考虑针对这些海图以外接长方形形式预存覆盖范围，提高索引精度。

矢量地图裁切，ENU转换优化算法

动机：

在筛选出候选海图后，需要将海图有关信息合并，按照任务区域裁切，并转换为ENU坐标形式。

直接实现与局限：

考虑到ENU坐标的转换是一个非线性的转换，且任务区域输入为视野范围（本船xx米范围内正方形区域）。因此为了保证裁切后边界精准，一种直接的思路是在合并海图的障碍物数据后，先障碍物转换ENU坐标，再按照任务区域进行裁切。此时性能瓶颈为ENU坐标转换（单进程方案最多需要数秒时间），原因为海图信息丰富，多边形有大量特征点需要转换坐标。

一种直接的思路是使用多进程池来并行处理转换需求，但是经实际测试，性能表现几乎没有提升，甚至倒退。分析如下：

1. 默认多进程任务负载划分以点转换任务或多边形转换任务划分，单个坐标点的转换实际耗费较小，但点的数量巨大，导致进程通信开销大，盖过了多进程的效率收益
2. 海图种类不同，有的是细碎多边形多，有的是少量复杂多边形，因此很难找到一种较为通用的任务负载划分算法，开发这样的算法所需开发工作量巨大

算法设计思路：

削减总体转换工作量，先裁切任务区域，再转换ENU坐标。这样就算单进程处理，由于总任务量的减少，转换时间也可大幅减少。

为了保证区域四角精准，开发ENU到经纬度的逆转换函数，计算四角经纬度作为依据进行裁切。流程可参考本节对应配图。

算法性能：

测试机上，优化后的算法，索引+合并+裁切+转换+栅格地图发布，在10km左右任务区域尺度上，总运行时间约为1秒（转换的时间复杂度为 $O(n^2)$ ），优化直接方案，满足工程要求。

Launch文件设计

在ROS2框架中，launch文件作为系统启动的核心配置工具，承担着自动化节点管理与运行环境部署的关键角色。通过基于Python的声明式语法，launch文件能够统一启动多个分散的节点（Node）、配置参数（Parameter）、设置话题重映射（Remapping）及命名空间（Namespace），并支持条件触发、事件响应等动态控制逻辑。其优势在于将复杂的多节点协作、硬件依赖管理及运行配置抽象为可复用的脚本，显著简化了机器人系统的初始化流程。开发者可通过单一指令快速激活完整的功能模块集群，同时保障跨平台部署的一致性，成为ROS2实现模块化、高可维护性系统架构的重要支撑。

在本项目中，预置了两个Launch文件。其中 `all_nodes.launch.py` 用于一键启动全部软件功能，指令运行顺序如下：

1. 启动ais_parse_node
2. 启动nmea_topic_driver（第三方）
3. 启动ais_db_node
4. 启动ais_tf_node
5. 启动ais_map_pub_node

`setup_test.launch.py` 用于调取历史回放数据，一键搭建离线开发/测试环境，指令运行顺序如下：

1. 启动 `ros2 bag play`
2. 启动nmea_topic_driver（第三方）
3. 启动ais_db_node
4. 启动ais_tf_node